ByCAN: Reverse Engineering Controller Area Network (CAN) Messages From Bit to Byte Level

Xiaojie Lin, Baihe Ma, Xu Wang, *Member, IEEE*, Guangsheng Yu, Ying He, *Senior Member, IEEE*, Ren Ping Liu, *Senior Member, IEEE*, and Wei Ni, *Fellow, IEEE*

***Abstract*—As the primary standard protocol for modern cars, the controller area network (CAN) is a critical research target for automotive cybersecurity threats and autonomous applications. As the decoding specification of CAN is a proprietary black-box maintained by original equipment manufacturers (OEMs), conducting related research and industry developments can be challenging without a comprehensive understanding of the meaning of CAN messages. In this article, we propose a fully automated reverse-engineering system, named ByCAN, to reverse engineer CAN messages. ByCAN outperforms the existing research by introducing byte-level clusters and integrating multiple features at both the byte and bit levels. ByCAN employs the clustering and template matching algorithms to automatically decode the specifications of CAN frames without the need for prior knowledge. Experimental results demonstrate that ByCAN achieves high accuracy in slicing and labeling performance, i.e., the identification of CAN signal boundaries and labels. In the experiments, ByCAN achieves slicing accuracy of 80.21%, slicing coverage of 95.21%, and labeling accuracy of 68.72% for the general labels when analysing the real-world CAN frames.**

***Index Terms*—Controller area network (CAN), in-vehicle network (IVNs), reverse engineering (RE), vehicular networks.**

I. Introduction

MASSIVE volumes of data are now generated and transmitted via in-vehicle networks (IVNs) as modern cars are equipped with more in-vehicle electronic control units (ECUs) with communication capabilities [1], [2]. The controller area network (CAN) protocol was first developed by Bosch in the 1980s [1] and serves as the de facto standard protocol for connecting ECUs embedded in cars [3], [4], [5]. The standard structure of the CAN frame is composed of the Start of Frame (SOF), arbitration field, control field, data field, cyclic redundancy check (CRC) field, ACK field, and End of Frame (EOF) as shown in Fig. 1. While the CAN protocol has a standardized frame structure, understanding the protocol's utilization for signal transmission remains challenging. This is because original equipment manufacturers (OEMs) encode the signals within the CAN frames' data fields (the data payloads) in proprietary ways that vary among OEMs, vehicle models, and years [6].

Fig. 1. Standard CAN frame format: 1-bit SOF, 11-bit ID, 1-bit remote transmission request (RTR), 1-bit identifier extension bit (IDE), 1-bit Reverse R0, 4-bit data length code (DLC), 0 to 8 byte data payload, 15-bit CRC, 1-bit CRC delimiter, 1-bit acknowledge (ACK), 1-bit ACK delimiter, and 7-bit EOF.

Understanding and decoding the data payloads of CAN frames is the first step to extracting the essential information to develop autonomous applications or explore automotive cybersecurity threats, such as vehicle location detection [7], [8], [9], [10]. Many studies have been dedicated to developing an automated reverse engineering (RE) system for CAN with minimum human interactions to uncover the communication patterns of CAN signals and deduce the decoding specification of the data payloads of CAN frames [11], [12], [13], [14], [15], [16], [17], [18], [19]. This could allow penetration testing and digital forensics on the CAN bus. Automotive insurance companies benefit from the RE systems because the result of those systems provides a way to evaluate cybersecurity risks as well as to discover driver fingerprinting for usage-based insurance [6].

Existing methodologies leverage CAN frames and additional data sources, such as the global positioning system (GPS) [12], inertial measurement unit (IMU) [16], database container (DBC) files [17], and on-board diagnostics II (OBD-II) data [20], to decode the specification of CAN signals. Among these, the OBD-II diagnostic data is easy to access via the OBD-II port, as all the modern cars are equipped with the OBD-II diagnostic system. OBD-II diagnostic data can be converted into human-readable accurate vehicle data with public formulas to be used in the matching process for associating semantic meanings with CAN signals. Both OBD-II diagnostic data and regular CAN frames can be collected from the OBD-II port. The RE systems can leverage both the CAN and OBD-II diagnostic data to create a comprehensive data set for RE purposes, eliminating the need for additional measurement equipment like IMUs.

Manuscript received 24 May 2024; revised 3 July 2024; accepted 26 July 2024. Date of publication 29 July 2024; date of current version 24 October 2024. This work was supported in part by the University of Technology Sydney, Insurance Australia Group (IAG), and in part by the iMOVE CRC (No. 5-028: Reverse Engineering Approaches to Vehicular Networks). *(Corresponding author: Xiaojie Lin.)*

Xiaojie Lin, Baihe Ma, Xu Wang, Ying He, and Ren Ping Liu are with the Global Big Data Technologies Centre, University of Technology Sydney, Sydney, NSW 2007, Australia (e-mail: xiaojie.lin@uts.edu.au; baihe.ma@uts.edu.au; xu.wang@uts.edu.au; ying.he@uts.edu.au; renping.liu@uts.edu.au).

Guangsheng Yu and Wei Ni are with the Commonwealth Scientific and Industrial Research Organization, Data61, Sydney, NSW 2122, Australia (e-mail: saber.yu@data61.csiro.au; wei.ni@data61.csiro.au).

Digital Object Identifier 10.1109/JIOT.2024.3435833

2327-4662 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

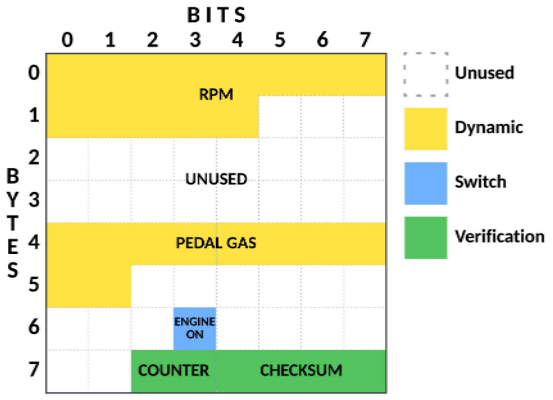

Fig. 2. Sample DBC: Mazda 3, Year 2019, and CAN ID of 0×01A [21]. CAN signals may occupy more than 1 byte, e.g., engine speed and pedal gas, which can be found by the yellow *Dynamic* CAN signals that take more than 8 bits consecutively. CAN signals often align with whole-byte offsets, such as the *Dynamic* signals aligned to the left and the *Verification* signals aligned to the right.

The primary objective of a CAN RE system is to identify the boundaries of CAN signals within a CAN frame payload field and to determine the associated signal labels. CAN messages are structured into frames, and the CAN frames of different CAN IDs have different lengths of the data payload. The payload servers as a container for multiple CAN signals, each with distinct bit positions and lengths as illustrated in Fig. 2. Before labeling CAN signals, the CAN signal slicing process first identifies the CAN signals' boundaries. The bit-flip rate, representing the frequency at which bits change from 1 to 0 or vice versa, is the primary feature in the existing methodologies for slicing CAN signals [6]. However, CAN signal slicing with only the bit-flip rate cannot precisely differentiate varying CAN signals. For example, distinguishing between *Dynamic* (e.g., engine speed signals) and *Verification* signals (i.e., *Checksum* and *Counter*) is challenging because their bit-flip rates are similar. By relying solely on the bit-level features [13], [14], [15], [16], [17], [18], [19], existing RE systems can result in excessive slicing of CAN signals.

The byte-level patterns of CAN signals are considered in this article based on the observation of DBC files from the OpenDBC repository [21]. The DBC files from the OpenDBC repository cover 82 cars, ranging from 2001 to 2023 for 19 distinct OEMs, such as Honda, Mazda, Mercedes Benz, and Toyota. There are 34 495 CAN signals covered in the DBC files from the OpenDBC, within which 98.25% of CAN signals are shorter than or equal to 16 bits of signal length. In summary, we found the patterns of CAN signals as follows.

1) *Pattern 1:* CAN signals usually occupy consecutive bits within a CAN frame, necessitating the signal slicing to locate the CAN signals.
2) *Pattern 2:* CAN signals may occupy more than 1 byte, highlighting the need for identifying signal boundaries at the byte level.
3) *Pattern 3:* CAN signals often align with whole-byte offsets, suggesting the advantage of initial byte-level slicing.

These insights are indicated in Fig. 2. Thereby, the byte-level clustering step can help in finding the boundaries of CAN signals at the byte level before slicing CAN signals at the bit level to enhance accuracy.

This article introduces a fully automated RE system named ByCAN to decode the data payloads of CAN frames using only the collected CAN frames and OBD-II diagnostic data with clustering-based matching learning and template matching algorithms. Unlike the previous studies, ByCAN processes CAN signals at both the byte and bit levels to reduce inaccurate in signal slicing, particularly, to avoid the issue of excessively sliced signal boundaries. To better capture the identified three patterns, ByCAN proposes multiple features, such as flip rate, average value, and distinct value ratio at both the byte and bit levels. The system's performance is assessed on the CAN messages captured from the real-world vehicles. The key contributions of the proposed ByCAN are summarized as follows.

1) We design a fully automated RE system using easily accessible data sources, i.e., CAN frames and OBD-II diagnostic data to decode the specifications of CAN signals.
2) We propose new features of CAN signals at both byte and bit-block levels to capture high-level patterns of CAN signals. Compared to the restrictive single-bit features, the byte and bit-block level features can capture the dynamics across a broader range of values, revealing complex and subtle patterns in CAN signals. To the best of our knowledge, this is the first attempt to use byte-level clusters and features to slice the CAN signals.
3) We propose a new slicing algorithm featuring density-based spatial clustering of applications with noise (DBSCAN), which eliminates the need for an estimated count of CAN signals. We also introduce a new template matching process utilizing dynamic time warping (DTW), specifically designed to align the large-volume CAN signals with the significantly less frequent OBD-II diagnostic data.

The remainder of this article is organized as follows. Section II introduces the relevant background information. Section III gives the related works. Section IV presents the proposed system. Section V evaluates the proposed system. Section VI concludes this article.

II. Background

A. Controller Area Network

A CAN frame contains the SOF, arbitration field, control field, data field, CRC field, ACK field, and EOF as in Fig. 1. ECUs have their own CAN IDs as unique identifiers [1]. The CAN ID of each frame is the reference to arbitrate the priority of concurrent CAN frames. Real-time information of vehicle states (e.g., vehicle speed, indicator status, and checksum) is enclosed in the data payload. The length of the data payload is up to 8 bytes. One CAN frame may contain multiple CAN signals of different functions in the data payload. For instance, different CAN signals related to doors and trunks (e.g., front left door open, rear right door open, and trunk open) can be located in the same CAN frame. OEMs have different

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

decoding specifications of CAN frames such that different car models may use different CAN IDs for the same CAN signals.

B. OBD-II Diagnostic Data

OBD-II is an automotive diagnostic system developed in 1992 and installed mandatory for all the cars built after 1996 in the U.S. [22]. The primary function of OBD-II diagnostic messages is to monitor the vehicle health state and enable the vehicle to communicate its operational information through the standardized diagnostic trouble codes (DTCs) [23]. OBD-II diagnostic data use parameter IDs (PIDs) to specify different vehicle states be requested or transmitted. These PIDs govern a range of data from the vehicle speed to sensor data for the fuel system status with a published standard to interpret the related signal messages. The same vehicle state, e.g., vehicle speed, can be obtained from the regular CAN frames and the requested OBD-II diagnostic data but following the CAN encoding and OBD-II diagnostic encoding, respectively.

C. Collecting CAN Frames and OBD-II Diagnostic Data

CAN frames and OBD-II diagnostic data are accessed via the OBD-II port that typically located under the dashboard. By connecting CAN analysers to the OBD-II port, CAN frames can be recorded passively, and meanwhile OBD-II diagnostic data can be requested actively. The CAN frames from the OBD-II port follow the same format as those over the in-vehicle CAN bus. This is because the OBD-II port is hard-wired to the CAN bus and provides CAN high and CAN low for passive listening. There could be a CAN gateway between CAN bus and OBD-II port which regulates the transfer of CAN data and can filter out the CAN frames based on specific CAN IDs before sending through the OBD-II port [24], [25]. Different methods can be used to gain direct access to the CAN bus, e.g., via the instrument pack or the rear of the radio, to bypass the CAN gateway and obtain all the CAN signals. The only difference between the CAN frames collected from the OBD-II port and those from the CAN bus wires is the number of CAN IDs and CAN signals in the collected data set. As existing RE systems focus on analysing CAN IDs independently, the absence of specific CAN IDs does not impede the RE process on the other accessible CAN IDs. Since, the OBD-II port is the common access point without damaging cars, most existing works promise utility and applicability by collecting the CAN frames via the OBD-II port.

D. CAN Database Container

The CAN DBC is a widely adopted format developed for the storage and interchange of the decoding specifications of CAN frames' data payload. DBC files are in the ASCII format to keep it human-readable and easy to manipulate programmatically. The DBC file describes the CAN IDs, the lengths of the data payloads of CAN frames, and the names of CAN signals. The DBC file also encapsulates the signal details, including the names, bit start position, length, endianness, scale, offset, range, and unit of CAN signals associated with CAN ID. However, DBC files vary across different vehicle models and makers [26]. OEMs rarely release DBC files due to security concerns, which makes the decoding specification of the data payloads of CAN frames a black box [27].

E. DBSCAN Clustering Algorithm

DBSCAN clustering algorithm was first introduced in 1996 and widely used to recognize data patterns [28]. DBSCAN identifies the core points with many nearby neighbors, the border points that are reachable from the core points but not dense themselves, and the outliers that lie alone in low-density regions [29]. Unlike the K-means, DBSCAN does not require the specification of the number of clusters as an input parameter. Instead, the DBSCAN clustering algorithm only requires two parameters: 1) the search radius ϵ around each point to find neighbors and 2) the minimum number of points *minPts* to determine a dense region for the core point. The number of clusters is determined by ϵ, *minPts* and the data itself, which makes DBSCAN particularly useful for the clustering tasks with unknown data structures.

III. Related Work

CAN RE systems deduce CAN signals' boundaries and labels by slicing and labeling CAN signals from the collected CAN frames [6]. Existing works have developed from the manual systems [30], [31], [32], which require significant human effort to the automated systems that analyse CAN signal patterns from the collected CAN frames for more efficient slicing and labeling [14], [15], [16], [17], [18], [19], [33], [34], [35], [36]. Markovitz and Wool are the first to develop an automated RE system and evaluate the performance with the simulated CAN frames [14]. Subsequently, Marchetti and Stabili [15] developed the READ system and evaluate the performance with the real-world cars. Machine learning algorithms are used to enhance the automated RE when analysing the real-world traffic trace of CAN frames where most CAN signals are transmitted at millisecond frequencies [37].

The bit-level features, such as the bit-flip rates of CAN signals are largely utilized in the existing automated CAN RE system. The RE system named READ calculates the bit-flip rate for each bit in the CAN frame's data payload and builds up the magnitude array to slice the CAN signals [15]. READ identifies CAN signals' boundaries by searching the magnitude array for pairs of consecutive bits where the first exhibits a higher bit-flip magnitude than the second, marking such instances as potential signal boundaries. Based on READ, Pesé et al. [16] used the bit-flip rate and magnitude array to determine CAN signals' boundaries but introduce a specific percentage threshold for the bit-flip rate decrease to refine the pair identification. The CANMatch system focuses on the bit-flip rate only but extends the use of bit-flip rate by incorporating the endianness of CAN signals at the byte level [17]. In [18], the CAN-D system is developed to slice and label CAN signals with the input of multiple bit-level features, e.g., the bit-flip rate, two-bit distributions, and entropy. In [26], the value of each byte in the CAN frame of different CAN IDs is used to find the clusters of various vehicle functions.

Existing RE systems utilize byte values or endianness merely as supplementary to bit-level slicing, thus neglecting additional byte-level patterns of CAN signals. The byte-level

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF RE SYSTEMS

	Approaches	Input Features of CAN Message		Data Source
		Bit-level	Byte-level	
READ [15]	Taxonomy	Bit-flip Rate	-	CAN
LibreCAN [16]	Taxonomy	Bit-flip Rate	-	CAN, GPS, IMU, OBD-II
CANMatch [17]	Unsupervised Learning, Frame Matching[1]	Bit-flip Rate	-	CAN, DBC Files
CAN-D [18]	Supervised Learning, Unsupervised Learning	Bit-flip Rate, Neighbouring Bit's Values and Flips[2]	-	CAN, OBD-II
Our work	Unsupervised Learning, Template Matching	Bit-flip Rate, Bit-block Flip Rate, Average Bit/Bit-block Value, Distinct Bit-block Value Ratio	Byte Flip Rate, Average Byte Value, Distinct Byte Value Ratio	CAN, OBD-II

[1] CAN signals whose ID can be matched in DBC files (ground truth) are directly reverse-engineered by using the ground truth results, which makes CANMatch largely depend on the ground truth.
[2] The neighboring bit's values and flip rates are not directly used but encoded.

features indicate different patterns of various CAN signals [38], [39]. According to our observation, CAN signals typically occupy up to 2 bytes, and those CAN signals associated with similar vehicle functions often consecutively reside within 1 or 2 bytes. By examining the byte-level patterns, such as the flip rate and average value of the byte, CAN signals can be initially separated at the byte level to prevent excessive slicing.

The existing works and the proposed ByCAN system are compared in Table I. The RE systems have developed from the basic taxonomy approaches to the machine learning-based methods. We apply unsupervised learning and template matching in the proposed ByCAN system for clustering and labeling steps, respectively. In prior works, the bit flip rate [15], [16], [17], [18] or the set of encoded features of neighboring bit's values and flips [18] are used to slice and label the CAN signals. In contrast to the existing methods, which have typically overlooked the byte-level patterns of CAN signals, ByCAN highlights the byte-level patterns and applies unsupervised learning algorithms in both the slicing and labeling procedures. ByCAN introduces multiple byte-level and bit-level features for the byte-level signal clustering and bit-level signal slicing. Additional data sources, such as GPS [16], DBC file [17], or OBD-II diagnostic data [18], are examined in the existing works to enhance the system performance. The proposed system only requires CAN frames and the request for the OBD-II diagnostic data that can be commonly collected from cars directly. No prior knowledge, such as the DBC file used in CANMatch [17] is required in ByCAN to slice and label the CAN signals, which ensures the applicability of the proposed system for the unknown cars and new models of cars. The slicing and labeling performance of ByCAN is evaluated with CAN frames collected from the real-world cars.

IV. PROPOSED SYSTEM

The proposed CAN RE system, ByCAN is demonstrated in Fig. 3. To the best of our knowledge, ByCAN is the first to utilize multiple features at both the byte and bit levels to identify the CAN signals' boundaries and labels. The introduced features capture the characteristics of CAN signals and enhance the performance of the proposed ByCAN. ByCAN consists of three procedures: 1) data preprocessing; 2) signal slicing; and 3) signal labeling.

A. Features Identification

Based on the empirical observations of the ground-truth DBC files and the collected CAN traces, the proposed system slices and labels CAN signals as *Unused*, *Switch*, *Dynamic*, and *Verification* types. The *Unused* CAN signals stand for the bits that are not taken or the values of the CAN signals are not changed during the communication period. The *Switch* CAN signals stand for the CAN signals that represent vehicle states whose values change rarely during the whole communication period. The *Dynamic* CAN signals stand for the vehicle's kinematic states, such as vehicle speed, which are quite active signals. The *Verification* CAN signals stand for the CAN signal counters or checksum that keep changing for every transmitted CAN frame.

Unlike the previous works that primarily focus on the bit-flip rate or other bit-level features, our system comprehensively analyses both the byte and bit characteristics. According to the characteristics of different CAN signals, we propose new features to distinctly categorize CAN signals, including the flip rate, the average value, and the distinct value ratio for each byte, bit, and bit-block as summarized below.

1) Flip Rate: The flip rate quantifies the variability of CAN signals across a sequence of CAN frames, which is fundamental for detecting the clusters of CAN signals at the byte or bit level within the data payloads of the CAN frames. Clusters with *Dynamic* or *Verification* CAN signals have high bit-flip rates, bit-block flip rates, and byte-flip rates. *Dynamic* CAN signals typically represent continuous variables, either incrementing or decrementing by the base unit, whereas *Verification* CAN signals pertain to discrete variables. This is because *Dynamic* CAN signals capture vehicle kinematic states, which consistently change while driving. Consequently, the *Verification* CAN signals are updated with each CAN frame.

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

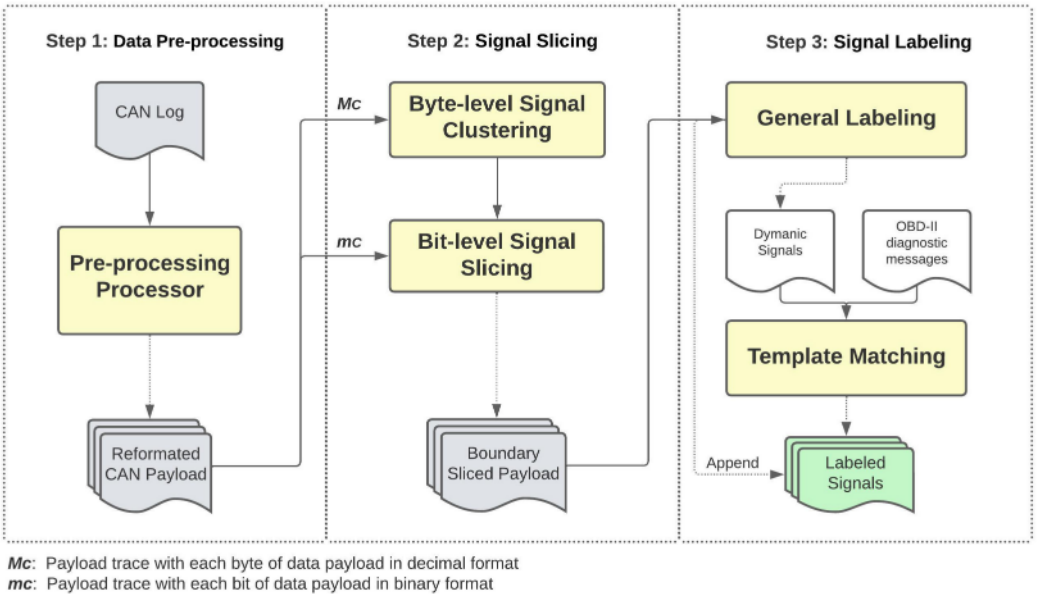

Fig. 3. ByCAN system: In the data preprocessing procedure, CAN messages are grouped by CAN ID first. Then, the grouped CAN messages are reformatted into the trace M_C and m_C with CAN frames' data payloads segmented at byte and bit level, respectively. In the signal-slicing procedure, the byte-level CAN signal features are extracted to deduce byte-level signal clusters. The bit-level CAN signal boundaries are further sliced within each byte-level cluster using proposed signal features at the bit level. In the signal labeling procedure, the sliced CAN signals are first labeled as general categories (i.e., *Unused*, *Switch*, *Dynamic*, and *Verification*). Finally, the descriptive labels are identified by applying the template matching algorithm to measure the similarity between the dynamic signals and OBD-II diagnostic messages.

2) Average Value: The use of the average value as a clustering parameter for CAN signals is substantiated by the observation that different types of CAN signals exhibit distinct patterns in their average values. For example, the average value of vehicle speed is usually between 0 and 100 km/h, while the average value of engine speed usually ranges between 600 and 5000 RPM. Therefore, the average value of the CAN signals could reveal the semantic meaning of the CAN signals.

3) Distinct Value Ratio: By measuring the number of values that are utilized actively, the distinct value ratios indicate the operational characteristics of the CAN signals, such as frequency of change and data density. The distinct value ratios indicate the functional complexity and utilization of the byte or the bit-block, revealing whether the byte or bit-block is predominantly occupied by dynamic values, static values, or a combination thereof. *Dynamic* CAN signals may vary within a range constrained by vehicle kinematic states, such as vehicle speed, while the verification signals can assume all the possible values as seen with the counter signals. Thus, *Verification* CAN signals have more distinct values than *Dynamic* CAN signals. Since, *Switch* CAN signals only have a few values to represent different states, *Switch* CAN signals have lower ratios of distinct values than *Dynamic* and *Verification* CAN signals. *Unused* signals have the lowest distinct byte-value ratio and distinct bit-block value ratio.

By leveraging these identified features, the proposed ByCAN system achieves enhanced performance and accuracy in distinguishing different CAN signal types. The comprehensive suite of features at both the byte and bit levels captures the distinct characteristics and patterns of each signal type more effectively.

B. Data Preprocessing

The initial phase of ByCAN is the data preprocessing of the collected CAN traces that include a variety of CAN IDs. The first step is to group CAN frames by CAN IDs due to the fact that CAN frames associated with distinct CAN IDs exhibit unique transmission and signal patterns. Subsequently, the CAN frame's data payload is segmented at the byte level and bit level, respectively. This conversion facilitates the subsequent stages of the byte-level and bit-level CAN signal slicing and labeling.

1) CAN Frame Grouping: The standard CAN frame has the SOF, arbitration field, control field, data field, CRC field, ACK field, and EOF as illustrated in Fig. 1. In practice, CAN frames are recorded and collected by CAN bus analysers[1] as CAN trace. Except for the essential data payload of the CAN frame, additional information, such as timestamps, CAN bus channel, and frame state is covered in the recorded CAN trace by the CAN bus analyser. The timestamp is crucial for analysing the signal patterns over time. All active functions of the targeted vehicle are represented in the data payloads of CAN frames, where the CAN signals vary across occupied bytes and value ranges. Different CAN IDs have different transmission patterns and CAN signal patterns for their CAN frames. For example, the data payload of CAN ID 0x094 could occupy 8 bytes for the active signals, whereas the data payload of CAN ID 0x01AB may only have 3 bytes used by the CAN signals. The first step of the data preprocessing procedure is to group CAN frames in the recorded CAN trace by their respective CAN ID C before slicing the CAN signals within the CAN frames.

[1]CAN bus analyzer can be used to view and log received and transmitted frames from the CAN bus. https://www.influxbigdata.in/post/can-bus-analyzer-choose-the-right-one.

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

TABLE II
NOTATIONS

Notation	Description
M_C/m_C	CAN trace whose frame payload is segmented by byte/bit for CAN ID C
T_C	The number of entries in M_C or m_C
Γ_i^C	i-th byte of the data payload in the CAN frame for CAN ID C
$\Gamma_{i,i+1}^C$	Sliced byte block with the i-th and $(i+1)$-th byte in the data payload of the CAN frame for CAN ID C
γ_k^C	k-th bit of the data payload in the CAN frame for CAN ID C
γ_{mn}^C	Sliced CAN signal from m-th to n-th bit of the data payload in the CAN frame for CAN ID C
$A_i^C/a_k^C/a_{mn}^C$	Average byte/bit/bit-block value of $\Gamma_i^C/\gamma_k^C/\gamma_{mn}^C$
$B_i^C/b_k^C/b_{mn}^C$	Flip rate of $\Gamma_i^C/\gamma_k^C/\gamma_{mn}^C$
V_i^C/v_{mn}^C	Distinct value set of Γ_i^C/γ_{mn}^C
U_i^C/u_{mn}^C	Distinct value ratio of Γ_i^C/γ_{mn}^C
θ_{mn}^C	Labeling parameter of γ_{mn}^C
g	Bit-level labeling function
β/ϕ	Flip rate function of Γ_i^C/γ_{mn}^C
$\zeta/\ \varpi$	Slicing accuracy/coverage
ξ	Labeling accuracy

2) Payload Reformatting: The collected CAN trace is split into multiple CAN traces for different CAN IDs. The trace of CAN ID C has T_C rows of CAN frames. The trace of CAN ID C is further converted into two traces, i.e., M_C and m_C, from the byte-level and bit-level perspectives, respectively. In M_C, a 64-bit CAN frame's data payload is converted into eight numerical values, with each byte represented by one value. The bit-level trace m_C retains the 64-bit representation of the CAN frame's data payload. Different from the existing works, the byte-level trace M_C is introduced to support our new byte-level processing, e.g., byte-level clustering. The bit-level trace m_C is used in the subsequent bit-level processing, e.g., bit-level slicing.

C. Signal Slicing

According to the identified CAN signal Pattern 1, the signal slicing is required to extract different CAN signals from the CAN frames. CAN signal boundaries are identified through a two-step process: 1) the byte-level signal clustering and 2) the bit-level signal slicing. Initially, the byte-level features extracted from the CAN frames are used to cluster the CAN signals at the byte level, establishing preliminary signal delimiters. Then, within each byte-level cluster, the boundaries of CAN signals are further sliced and defined at the bit level. The notations used are detailed in Table II. Algorithm 1 illustrates the CAN signal slicing at both the byte and bit levels.

1) Byte-Level Signal Clustering: The processed CAN trace M_C of CAN ID C whose data payload segmented by bytes is used in this step. The byte-level features are extracted from the trace M_C for each byte in the data payload. Γ_i^C, $1 \le i \le 8$, represents the ith byte of the data payload of the CAN frame for CAN ID C.

Algorithm 1: Slicing Algorithm

Input : CAN trace M_C and m_C for CAN ID C
Output: Sliced CAN signal γ_{mn}^C

```
/* Identify byte-level features of M_C.                    */
1 Compute byte-level features B_i^C, A_i^C, and U_i^C;   ▷
  with (1), (2) and (3)
/* Cluster byte blocks of M_C by applying the
   clustering algorithms.                                  */
2 Γ_i^C or Γ_{i,i+1}^C ← DBSCAN_Cluster(M_C, B_i^C, A_i^C, U_i^C);
/* Slice bit-level signal boundary within byte-level
   blocks.                                                 */
3 for γ_k^C in Γ_i^C or Γ_{i,i+1}^C do
4     Compute the bit-flip rate b_k^C;   ▷ with (4);
5     Compute the average bit value a_k^C;   ▷ with (5);
6 end
/* Cluster bit blocks of Γ_i^C or Γ_{i,i+1}^C with m_C by
   applying the clustering algorithms.                     */
7 return γ_mn^C ← DBSCAN_Cluster(m_C, b_k^C, a_k^C);
```

According to the identified Patterns 2 and 3, CAN signals can be first located at the byte level to enhance the slicing performance. ByCAN proposes the new byte-level features, i.e., the byte-flip rate B_i^C, the average byte value A_i^C, and the distinct byte-value ratio U_i^C. B_i^C represents the frequency at which the value of Γ_i^C changes. A_i^C is the mean value of Γ_i^C and U_i^C indicates the quantity of different values for Γ_i^C. Different CAN signals distinguish from each other with different features of B_i^C, A_i^C, and U_i^C.

The byte-flip rate of the Γ_i^C, denoted by B_i^C, as given by

$$B_i^C = \frac{\sum_{j=1}^{T_C-1} \beta(M_C, i, j)}{T_C - 1}$$
$$\beta(M_C, i, j) = \begin{cases} 0, & \text{if } \Gamma_{i(j+1)} = \Gamma_{ij} \\ 1, & \text{if } \Gamma_{i(j+1)} \neq \Gamma_{ij} \end{cases} \tag{1}$$

where Γ_{ij} is the value of Γ_i^C in the jth entry of M_C, and $1 \le j \le T_C$. The byte-flip function $\beta(M_C, i, j)$ is 0, when Γ_i^C in the adjacent entries are the same and 1 otherwise. We can have $0 \le B_i^C \le 1$.

The average value of Γ_i^C, denoted by A_i^C, can be given by

$$A_i^C = \frac{\sum_{j=1}^{T_C} \Gamma_{ij}}{T_C}. \tag{2}$$

The distinct byte-value ratio of Γ_i^C, denoted by U_i^C, can be calculated by

$$U_i^C = \frac{|V_i^C|}{256}, \quad 1 \le \left|V_i^C\right| \le 256 \tag{3}$$

where V_i^C is the set of distinct values of Γ_i^C. $|V_i^C|$ gives the number of elements in V_i^C. The maximum number of $|V_i^C|$ for a byte is $2^8 = 256$, since 1 byte has 8 bits.

The proposed byte-level features are utilized to cluster the CAN signals at the byte level by applying the clustering algorithms as shown in Algorithm 1 line 2. The number of

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

CAN signals varies for different CAN ID and is unknown, which means the system cannot determine the number of clusters when slicing the CAN signals in a CAN frame either at the byte or bit level. Thus, ByCAN utilizes the DBSCAN clustering algorithm that does not require to specify the number of clusters in advance. Based on the observation of DBC files, CAN signals take up to 2 bytes position of a CAN frame. ByCAN restricts the cluster to be maximum 2 byte long and identifies byte blocks $\Gamma^C_{i,i+1}$ with 2 bytes and Γ^C_i with 1 byte.

2) Bit-Level Signal Slicing: The proposed system slices the bit-level CAN signal boundaries within the identified byte-level signal clusters with 8 or 16 bits.

The kth bit of the payload in the CAN frame of CAN ID C, denoted by γ^C_k, has $1 \leq k \leq 64$. The bit-flip rate of γ^C_k is denoted by b^C_k, as given by

$$b^C_k = \frac{\sum_{j=1}^{T_C-1} \gamma^i_{k(j+1)} \oplus \gamma^i_{kj}}{T_C - 1} \tag{4}$$

where k indicates the kth bit of the payload in the CAN frame of CAN ID C. The j denotes the jth CAN frame of m_C, $1 \leq j \leq T_C$. γ^i_{kj} is the value of γ^C_k in the jth entry of m_C.

The average bit value of γ^C_k, denoted by a^C_k, is as given by

$$a^C_k = \frac{\sum_{j=1}^{T_C} \gamma^i_{kj}}{T_C}. \tag{5}$$

The bit-flip rate b^C_k and average bit value a^C_k are used to cluster bit-level blocks γ^C_{mn}, $m \leq n$. The m and n of γ^C_{mn} are the start and the end bit positions in the data payload of the CAN frame for CAN ID C, respectively. Note that $m = n$ when γ_{mn} takes only 1 bit.

After identifying the byte-level and bit-level features, CAN signal clusters at both levels are generated using the DBSCAN clustering algorithm, as outlined in Algorithm 1. Under the clustered byte blocks, bit-level features are further calculated to cluster CAN signals at the bit level. The DBSCAN algorithm is particularly suited for the clustering process because it adeptly handles the variations in signal density, which is common in CAN frames. Additionally, the DBSCAN algorithm is suited for clustering signals by each CAN ID or each byte, as it does not require prior knowledge of the number of clusters—a common scenario in RE the data payloads of CAN frames.

D. Signal Labeling

The signal labeling is illustrated in Algorithm 2, including the general labeling and template matching process. Bit-level features of sliced CAN signal blocks from the signal slicing step are identified to associate the sliced CAN signals with the general labels. Due to the unknown numbers of CAN signal types, the DBSCAN algorithm is used to cluster based on the labeling thresholds, distinguishing different CAN signals' labels using the identified bit-level features. For the identified *Dynamic* CAN signals, the DTW template matching algorithm is further used to align them with the templates of the OBD-II diagnostic data. DTW effectively compensates for discrepancies, such as time differences, shifts, or offsets between the CAN signals and the corresponding templates of the OBD-II diagnostic data. This alignment is crucial, as the CAN frames typically transmit faster than the OBD-II diagnostic responses that lack a fixed correspondence ratio with the CAN frames.

Algorithm 2: Labeling Algorithm

Input : Sliced CAN signal γ^C_{mn}, CAN trace m_C for CAN ID C, time series of the templates of OBD-II diagnostic data S_x
Output: CAN signal label of γ^C_{mn}

```
/* Identify bit-level features of all γ^C_mn.                    */
Compute bit-level features b^C_mn, a^C_mn, and u^C_mn;   ▷ with (6), (7) and (8)
Compute labeling parameter θ^C_mn;   ▷ with (9)
/* Determine labeling thresholds ε_0 by applying clustering algorithms with θ^C_mn, b^C_mn and a^C_mn of all bit-blocks.   */
ε_0 ← Cluster(θ^C_mn, b^C_mn, a^C_mn);
foreach γ^C_mn do
    /* Assign general labels.                                    */
    Label γ^C_mn with g(θ^C_mn, b^C_mn);   ▷ with (10)
    /* Assign descriptive labels to Dynamic signals by using the template matching algorithms.   */
    if g(θ^C_mn, b^C_mn) = Dynamic then
        /* Convert the CAN message values of γ^C_mn into the time seires sequence E_x.   */
        E_x ← Serialize(γ^C_mn);
        γ^C_mn.TemplateMatch(S_x, E_x);
    end
end
return Labels of γ^C_mn;
```

1) General Labeling: The proposed system labels CAN signals as *Unused*, *Switch*, *Dynamic*, and *Verification*. The flip rate is the common parameter to distinguish and label different CAN signal types. The proposed system improves the labeling process by introducing the labeling parameter and combining it with the flip rate to label the CAN signals. To distinguish the signals with similar flip rates but different ratios of distinct values (e.g., *Unused* and *Switch*), ByCAN combines the flip rate and the ratio of distinct values into a new labeling parameter.

The bit-block flip rate of γ^C_{mn}, denoted as b^C_{mn}, presents the flip rate of sliced CAN signal blocks in the payload of a CAN frame for CAN ID C, as given by

$$b^C_{mn} = \frac{\sum_{j=1}^{T_C-1} \phi(m_C, i, mn, j)}{T_C - 1}$$
$$\phi(m_C, i, mn, j) = \begin{cases} 0, & \text{if } \gamma^i_{mn(j+1)} = \gamma^i_{mnj} \\ 1, & \text{if } \gamma^i_{mn(j+1)} \neq \gamma^i_{mnj} \end{cases} \tag{6}$$

where the value of γ^C_{mn} of the jth entry of m_C is denoted by γ^i_{mnj}, and $1 \leq j \leq T_C$. The bit flip function ϕ is 1 when γ^C_{mn} in the adjacent entries are different and 0 otherwise.

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

The average bit-block value of γ_{mn}^C, denoted by a_{mn}, can be calculated by

$$a_{mn}^C = \frac{\sum_{j=1}^{T_C} \gamma_{mnj}^i}{T_C}. \tag{7}$$

The distinct bit-block value ratio of γ_{mn}^C, denoted by u_{mn}^C, indicates the frequency of different values of a CAN signal in the CAN trace and is as given by

$$u_{mn}^C = \frac{|v_{mn}^C|}{2^{n-m+1}}, \quad 1 \le \left|v_{mn}^C\right| \le 2^{n-m+1} \tag{8}$$

where v_{mn}^C is the set of distinct bit-block values of γ_{mn}^C, and $|v_{mn}^C|$ is the size of the set v_{mn}^C. 2^{n-m+1} is the maximum number of the bit block taken up from the mth bit to the nth bit (m $\le$ n), i.e., the maximum number of $|v_{mn}^C|$.

The labeling parameter θ_{mn}^C is defined to label the sliced CAN signals γ_{mn}^C, as given by

$$\theta_{mn}^C = b_{mn}^C \times u_{mn}^C \tag{9}$$

where $0 \le \theta_{mn}^C \le 1$. The parameter θ_{mn}^C combines b_{mn}^C and u_{mn}^C to consider both the bit-block flip rate and the ratio of distinct bit-block values when labeling the CAN signals, thereby maximizing the dissimilarity of different labels. Instead of using u_{mn}^C only to label the CAN signals, θ_{mn}^C can distinguish the CAN signals with similar flip rates and different ratios of distinct values.

Unused CAN signal blocks have $\theta_{mn}^C = 0$ and $b_{mn}^C = 0$ by which the signal can be deduced as unchanged no matter the value it represents. *Verification*, *Switch*, and *Dynamic* CAN signals have $b_{mn}^C > 0$. *Switch* CAN signals are triggered less frequently than the *Dynamic* CAN signals. *Dynamic* and *Verification* CAN signals usually have higher b_{mn}^C. Given all the calculated θ_{mn}^C, the threshold ε_0 is determined by unsupervised clustering algorithms and used to label *Switch*, *Dynamic*, and *Verification* CAN signals. The bit-level labeling function g is given by

$$g\left(\theta_{mn}^C, b_{mn}^C\right) = \begin{cases} \text{Unused} & \text{if } \theta_{mn}^C = 0 \quad \text{and} \quad b_{mn}^C = 0 \\ \text{Switch} & \text{if } 0 < \theta_{mn}^C \le \varepsilon_0 \\ \text{Dynamic} & \text{if } \varepsilon_0 \le \theta_{mn}^C \quad \text{and} \quad b_{mn}^C < 0.99 \\ \text{Verification} & \text{if } \varepsilon_0 \le \theta_{mn}^C \quad \text{and} \quad b_{mn}^C \ge 0.99. \end{cases} \tag{10}$$

The labeling function g takes into account θ_{mn}^C and b_{mn}^C to match the label based on different CAN signals' features. *Unused* CAN signals have both θ_{mn}^C and b_{mn}^C that are equal to 0. The *Switch* CAN signals have lower θ_{mn}^C than the *Dynamic* and *Verification* CAN signals. The *Verification* CAN signals have higher θ_{mn}^C and the highest b_{mn}^C. The proposed system sets the threshold value for b_{mn}^C at 0.99 to distinguish between the *Verification* and *Dynamic* CAN signals. This threshold mitigates the uncommon scenario where the *Verification* CAN signals fail to toggle in each CAN frame within the recorded CAN trace.

2) Template Matching: OBD-II diagnostic data are used in this step since the information of some vehicle states, e.g., vehicle speed and engine speed, can be extracted from the OBD-II diagnostic data. ByCAN sends OBD-II diagnostic requests, such as the request of PID 0C for the engine speed state to the target vehicle. Following the public OBD-II diagnostic data standard,[2] the requested OBD-II diagnostic data are examined to translate and extract the values of the relevant states. The sliced and labeled *Dynamic* CAN signals are then matched with the extracted information from the OBD diagnostic response to further determine the descriptive signal labels without prior knowledge by leveraging template matching algorithms. The proposed system builds the matching template with the collected OBD-II diagnostic data that contain the real-time dynamic signal values, such as vehicle speed and accelerator pedal position. ByCAN associates sliced CAN signals with descriptive labels: speed, wheel angle, throttle pedal position, and brake pedal position related.

The generated templates represent the time series of corresponding signal values. S_x denotes a time series template of the OBD-II diagnostic data for one *Dynamic* CAN signal x, where $S_x = \{s_1, \ldots, s_a, \ldots, s_y\}$. ByCAN measures the similarity between the sliced *Dynamic* CAN signal candidates and the templates. The time series sequence of a sliced *Dynamic* CAN signal candidate is denoted as E_x, $E_x = \{e_1, \ldots, e_b, \ldots, e_z\}$. Note that, $z = T_C$. The values of E_x are the values of the labeled signal block γ_{mn} of m_C in sequence. The defined time interval of the OBD-II diagnostic data is different from the transition rate of different CAN IDs. Thus, length y of S_x may be longer or shorter than the length z of E_x.

In the proposed system, the DTW algorithm is used as the similarity measure, since S_x and E_x are the unequal-length time series [40]. A w-by-z matrix is constructed to align S_x and E_x using (11), where $d_{a,b}$ is the element (a and b) in the constructed w-by-z matrix to present the Euclidean distance between the points s_a and e_b (i.e., one for the *Dynamic* CAN signal and the other for the template of the OBD-II diagnostic data)

$$d_{a,b} = (s_a - e_b)^2. \tag{11}$$

The warping path W is given in (12), where the kth element $w_k = (a, b)_k$

$$\begin{aligned} W_x &= \{w_1, w_2, \ldots, w_k, \ldots, w_K\} \\ \max(y, z) &\le K < y + z - 1. \end{aligned} \tag{12}$$

The optimal path between S_x and E_x is determined by finding the minimum path from the set of all the admissible paths as given by

$$\text{DTW}(S_x, E_x) = \min\left(\sqrt{\sum_{k=1}^{K} w_k}\right) \tag{13}$$

where $\text{DTW}(S_x, E_x)$ is the optimal path (i.e., the shortest path in our case) for the *Dynamic* CAN signal candidate with all

[2]The standard OBD-II PIDs are defined by SAE J1979, which contains the formula to translate the OBD-II diagnostic response into meaningful data. https://en.wikipedia.org/wiki/OBD-II_PIDs.

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

the templates of the OBD-II diagnostic data. The proposed system measures the optimal paths for the signal candidate with all the assessed OBD-II diagnostic data templates. The descriptive label with the shortest path is finally associated with the signal candidate.

E. Assessment Criteria

To quantify the performance of ByCAN, we propose three metrics, i.e., slicing accuracy ζ, slicing coverage ϖ, and labeling accuracy ξ.

Slicing accuracy (ζ) indicates the percentage of correctly sliced CAN signal bits, is as given by

$$\zeta = \sum_{\mathrm{id}=1}^{x} \frac{S_{\mathrm{id}}^{\gamma_{mn}^{C}}}{G_{\mathrm{id}}^{\gamma_{mn}^{C}}} \tag{14}$$

where $S_{\mathrm{id}}^{\gamma_{mn}^{C}}$ is the number of bits that are correctly deduced CAN signal boundaries for γ_{mn}^{C} of the *id*th CAN ID frame. $G_{\mathrm{id}}^{\gamma_{mn}^{C}}$ is the number bits of exact signal boundaries for γ_{mn}^{C}. The x standards for the number of CAN IDs analysed in the system evaluation.

Slicing coverage (ϖ) represents the portion of sliced CAN signals that are correctly located within the ground truth CAN signal boundaries. ϖ can be calculated by

$$\varpi = \sum_{\mathrm{id}=1}^{x} \frac{\mathbb{S}_{\mathrm{id}}^{\gamma_{mn}^{C}}}{G_{\mathrm{id}}^{\gamma_{mn}^{C}}} \tag{15}$$

where x is the number of CAN IDs, and $\mathbb{S}_{\mathrm{id}}^{\gamma_{mn}^{C}}$ stands for the number of CAN signals that are located within the ground truth CAN signal boundaries of the *id*th CAN ID signal.

Labeling accuracy (ξ) stands for the system performance of labeling CAN signals with the general or the descriptive labels. ξ assesses the sliced CAN signals that are correctly labeled, as given by

$$\xi = \sum_{\mathrm{id}=1}^{x} \frac{L_{\mathrm{id}}^{\gamma_{mn}^{C}}}{\mathbb{G}_{\mathrm{id}}^{\gamma_{mn}^{C}}} \tag{16}$$

where $L_{\mathrm{id}}^{\gamma_{mn}^{C}}$ is the number of correct labeled CAN signals for γ_{mn}^{C} of the *id*th CAN ID frame. The label of γ_{mn} in the ground truth is determined by the label that takes up more bits in γ_{mn}. $\mathbb{G}_{\mathrm{id}}^{\gamma_{mn}^{C}}$ is the ground truth label of γ_{mn}.

V. Evaluation and Result

A. Experimental Setup

In our experiments, the real-world CAN traces were collected from the three cars of different makes and models: 1) the 2006 Mazda 3 Automatic; 2) the 2017 Honda Civic Manual; and 3) the 2022 Toyota RAV4 Hybrid. The PEAK PCAN-USB Pro[3] adapter was used as the CAN analyser in the experiment to listen and record CAN frames on the CAN bus via the OBD-II port. An OBD-II cable was connected between the OBD-II port of the target vehicle and the PEAK PCAN-USB Pro, and the PEAK PCAN-USB Pro was connected with the USB port of the experiment laptop to run the PCAN-Explorer 6[4] to record the CAN frames into trace files. The CAN frames were collected passively from the CAN bus, while the OBD-II diagnostic data were collected by sending related OBD-II requests. OBD-II diagnostic requests with the PIDs of 04 (calculated engine load), 0C (engine speed), 0D (vehicle speed), 11 (throttle position), 45 (relative throttle position), 47 (absolute throttle position B), 48 (absolute throttle position C), 49 (absolute throttle position D), 4A (absolute throttle position E), and 4B (absolute throttle position F) were sent to the target vehicle. Both CAN frames and OBD-II data were monitored and recorded into the trace files simultaneously.

The collected CAN trace files were converted into CSV files using the compatible software PEAK-Converter.[5] The parsed CSV files of the collected CAN trace files were fed into the ByCAN system. The byte-level features, i.e., the flip rate, the average value, the distinct byte-value ratio of byte, were extracted from the payload of recorded frames. ByCAN restricts the byte clusters to have a minimum length of 1 byte and a maximum length of 2 bytes. Within the identified byte clusters, the bit blocks of CAN signals were further clustered with the DBSCAN clustering algorithm with the bit-level features. We ran the data collection and static analysis with the DELL Latitude 5420 laptop, with 16 GB RAM and an 11th Gen Intel Core i7-1185G7 CPU.

All possible car functions, e.g., acceleration, deceleration, wheel steering, and car indicators were triggered to collect as many CAN signals as we could. The data collection step takes 2 min on average to collect the regular CAN frames and the OBD-II diagnostic data for ByCAN to reverse engineer CAN signals. The experimental results were then assessed with the ground truth from the OpenDBC repository [21]. Since, the DBC files from the OpenDBC were generated from manual RE, not all the CAN signals in the tested cars were covered in the DBC files from OpenDBC. Thus, only the CAN signals exist on both the collected CAN trace files and the ground truth DBC files were evaluated. The DBC files utilized in our experiments were cross-validated with laboratory vehicles by engineers at the Insurance Australia Group (IAG) Research Center, ensuring the reliability of our ground truth data. The evaluation focused on the system's slicing and labeling performance, examining the impact of CAN signal type, signal length, CAN IDs, and the number of CAN frames. The CAN traces collected from the individual cars were used as the input to the proposed system for different rounds of evaluations. The overview of the experimental results is the average performance of all the three cars tested in our experiments, which is evaluated and discussed in this section.

[3]PEAK PCAN-USB Pro is one of the CAN analysers bridging a PC with the CAN network to collect the CAN frames. https://www.peak-system.com/PCAN-USB-Pro-FD.366.0.html?&L=1.

[4]PEAK PCAN-Explorer 6 is the Windows system software compatible with the PEAK PCAN-USB Pro to receive, send, and record the CAN frames. https://www.peak-system.com/PCAN-Explorer-6.415.0.html?&L=1.

[5]PEAK PCAN-Converter is the Windows system software compatible with the PEAK PCAN-USB Pro to convert trace files to various output formats for the processing and analysis purpose. https://www.peak-system.com/PEAK-Converter.554.0.html?&L=1.

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

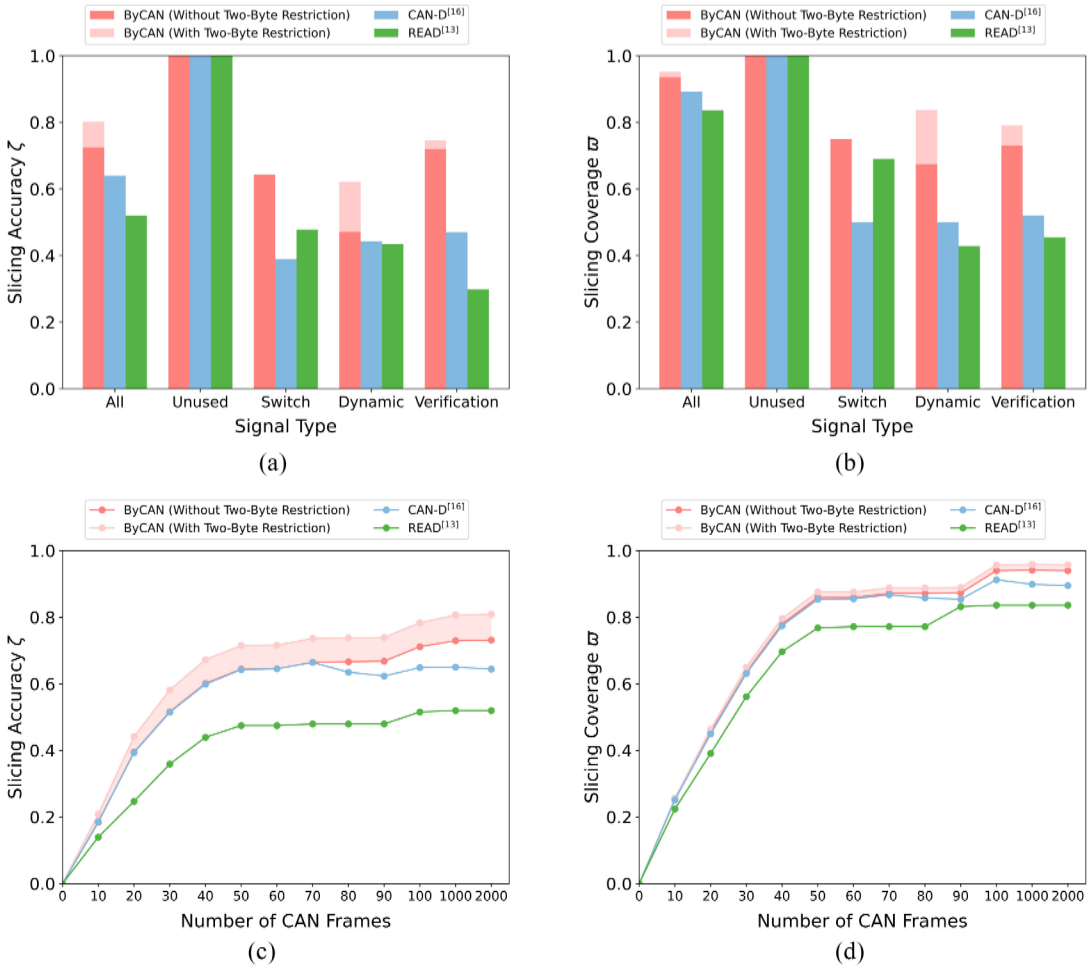

Fig. 4. Comparison of slicing accuracy and slicing coverage of different systems. The y-axis is the slicing accuracy ζ and the x-axis is CAN signal type in (a); the y-axis is the slicing coverage ϖ and the x-axis is CAN signal type in (b); the y-axis is the slicing accuracy ζ and the x-axis is the number of CAN frames in (c); and the y-axis is the slicing coverage ϖ and the x-axis is the number of CAN frames in (d). Note that, the *Verification* CAN signals represent both the *Counter* and *Checksum* signals.

TABLE III
COMPARISON OF OVERALL PERFORMANCE

System	Slicing accuracy ζ	Slicing coverage ϖ	Labeling accuracy ξ
ByCAN	**80.21%**	**95.21%**	**68.72%**
CAN-D [18]	63.88%	89.21%	67.62%
READ [15]	51.99%	83.61%	64.48%

B. Overall Performance

The proposed system ByCAN is compared against the two other RE systems, i.e., READ [15] and CAN-D [18], with the evaluation metrics comprising slicing accuracy (ζ), slicing coverage (ϖ), and labeling accuracy (ξ), as summarized in Table III. The READ system relies solely on the bit-flip rate to slice and label CAN signals. The CAN-D system leverages multiple bit-level features, including the bit-flip rate, two-bit distributions, and entropy. The proposed ByCAN system distinguishes the READ and CAN-D from the initial step of byte-level clustering before identifying bit-level signal boundaries within the byte clusters. The experimental results presented are the averaged performance metrics of evaluated systems with real-world CAN traces collected from all the tested cars included in our study.

In our experiments, ByCAN demonstrates a superior slicing accuracy ζ of 80.21%, which notably surpasses that of CAN-D and READ with slicing accuracy ζ of 63.88% and 51.99%, respectively. ByCAN achieves slicing coverage ϖ of 95.21%, which also outperforms the CAN-D (89.21%) and READ (83.61%). In terms of labeling accuracy ξ, ByCAN maintains the lead with a 68.72% labeling accuracy, while CAN-D and READ are behind with 67.62% and 64.48%, respectively. The experimental result indicates that the proposed ByCAN system gains a better overall performance in decoding CAN signal specifications without any prior knowledge.

C. Slicing Performance

1) Size of Byte-Level Clusters: According to Pattern 2 and our observation, most CAN signals take less than 16 bits in the CAN frames' data payloads. We compare the ByCAN's performance with and without the two-byte restriction in the byte-level clustering step, and ByCAN with the restriction has better overall performance, as shown in Fig. 4. The setting

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISON OF SLICING ACCURACY AND COVERAGE STATISTICS ACROSS CAN IDS

	System	Min	Max	Mean	Median	Variance	Standard Deviation
Slicing accuracy ζ	**ByCAN**	**48.43%**	**96.43%**	**80.93%**	**83.59%**	**0.0211**	**0.1453**
	CAN-D [18]	9.38%	96.43%	66.65%	80.73%	0.0949	0.3081
	READ [15]	6.25%	95.31%	54.47%	64.84%	0.1098	0.3314
Slicing coverage ϖ	**ByCAN**	**72.73%**	**100%**	**94.48%**	**96.41%**	**0.0053**	**0.0730**
	CAN-D [18]	27.27%	100%	81.19%	91.86%	0.0538	0.2320
	READ [15]	20.00%	98.31%	72.47%	84.53%	0.0646	0.2541

of the two-byte restriction for the byte-level clusters' length enhances the overall slicing performance of ByCAN from 72.47% to 80.21% for slicing accuracy ζ, and from 93.58% to 95.22% for slicing coverage ϖ, as demonstrated in Fig. 4(a) and (b). ByCAN with two-byte restriction also gains better slicing accuracy for *Dynamic* and *Verification* CAN signals, which is 15.03% and 2.62% higher than that without the two-byte restriction. In terms of the slicing coverage ϖ, the setting of two-byte restriction for the signal clustering increases ϖ from 67.44% to 83.72% for the *Dynamic* CAN signal, and from 73% to 79.07% for the *Verification* CAN signal. The reason is that the restriction of 2 byte on the signal cluster's length allocates CAN signals more accurately at the byte-level boundary. Since, the *Verification* and *Dynamic* CAN signals tend to take 1 or 2 bytes, the restriction setting improves the relevant slicing accuracy and coverage. As shown in Fig. 4(c) and (d), the ByCAN system with two-byte restriction also has better slicing accuracy and coverage than that without two-byte restriction across different numbers of CAN frames.

2) CAN Signal Type: When comparing the ByCAN system with two-byte restriction to the CAN-D and READ systems, ByCAN demonstrates higher slicing accuracy and coverage than the other two systems, regardless of the signal type as depicted in Fig. 4(a) and (b). ByCAN reaches up to a slicing accuracy ζ of 80.21% that is 16.21% higher than the CAN-D and 28.22% higher than the READ as shown in Fig. 4(a). The same trend can be found in Fig. 4(b), where ByCAN has the slicing coverage ϖ of 93.58% and the CAN-D and READ systems have that of 89.21% and 83.61%. ByCAN is superior in slicing the *Switch*, *Dynamic*, and *Verification* (i.e., *Counter* and *Checksum*) CAN signals as indicated in Fig. 4(a) and (b). The reason is that the *Switch* signals with similar functionalities and *Verification* signals tend to be located within 1 or 2 bytes, where the byte-level clustering of ByCAN enhances the slicing performance by avoiding the over-slicing at the bit level. Taking the advantage of byte-level clustering, ByCAN has a better slicing coverage ϖ than the CAN-D and READ systems for the *Dynamic* CAN signals that usually take 1 or 2 bytes long. For *Unused* CAN signals, all the systems can reach the 100% slicing accuracy and coverage because the *Unused* signals have a bit-flip rate that always equals 0.

3) Number of CAN Frames: As shown in Fig. 4(c) and (d), all the systems accurately slice a comparable number of CAN signals. Both ζ and ϖ increase with the number of CAN frames fed into the RE systems. However, the slicing performance converges when the number of CAN frames exceeds 1000, a point at which most functionalities are triggered in our experiments. The optimum value of T_C is considered as 1000 for both the slicing accuracy and coverage. When $T_C = 1000$, ByCAN with two-byte restriction ($\zeta = 80.90\%$) surpasses the CAN-D system ($\zeta = 65.03\%$) and the READ system ($\zeta = 51.99\%$) as demonstrated in Fig. 4(c). Although the slicing accuracy differs significantly among the systems, there is only a slight variation in the slicing coverage ϖ among them as illustrated in Fig. 4(d). When $T_C = 1000$, ByCAN with the two-byte restriction reaches 95.90% of ϖ while the CAN-D and READ systems achieve 89.89% and 83.61% of ϖ, respectively.

4) CAN IDs: As indicated in Table IV, the performance of the systems when slicing CAN signals shows variability among different CAN IDs. This variability can be attributed to the differences in the number and types of CAN signals contained within the frames of each CAN ID. Specifically, ByCAN exhibits slicing accuracy ζ ranging from 48.43% to 96.43%, signifying substantial performance variation tied to the specific CAN ID. By contrast, CAN-D and READ display a wider range (9.38% to 96.43% and 6.25% to 95.31%, respectively), of the slicing accuracy, indicating a broader disparity in performance across CAN IDs. Furthermore, while ByCAN achieves high slicing coverage ϖ with minimal variability as evidenced by the low variance of 0.0053, it still reveals a significant gap between its minimum and maximum coverage (72.73% to 100%). The CAN-D and READ systems have a higher variance (0.0538 and 0.0646) than ByCAN, underscoring the inconsistency in the slicing coverage across different CAN IDs. A notable spread in the slicing coverage is also demonstrated for CAN-D (27.27% to 100%) and READ (20.00% to 98.31%) in Table IV.

5) CAN Signal Length: The system performance of slicing CAN signals with different signal lengths is detailed in Table V. Given the diverse lengths of CAN signals, this article specifically focuses the commonly used bit lengths, namely 1 bit, 2 bits, 4 bits, 8 bits, 10 bits, and 16 bits - for the generality. It is important to note that *Unused* CAN signals, all considered as 1-bit long are excluded from this discussion. ByCAN demonstrates higher ζ and ϖ for CAN signals with lengths of 1 bit, 4 bits, 8 bits, 10 bits, and 16 bits. Notably, ByCAN has better performance for longer CAN signal lengths. For instance, ByCAN achieves slicing coverage ϖ of 81.25% for 16-bit long CAN signals, nearly twice the value obtained by the READ system at 43.75% for the same length. Specifically, ByCAN achieves a ζ of 81.52% and 70% for four-bit long CAN signals and attains a perfect ζ and ϖ (both 100%) for ten-bit long CAN signals.

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISON OF SLICING ACCURACY AND COVERAGE OF CAN SIGNAL LENGTH

	System	1 Bit[1]	2 Bits	4 Bits	8 Bits	10 Bits	16 Bits
Slicing accuracy ζ	ByCAN	**77.50%**	41.94%	**81.52%**	**45.63%**	**70%**	**59.38%**
	CAN-D [18]	57.50%	37.10%	56.52%	27.5%	35%	56.25%
	READ [15]	50%	**83.87%**	30.43%	25%	50%	43.75%
Slicing coverage ϖ	ByCAN	**70%**	48.39%	**100%**	**65%**	**100%**	**81.25%**
	CAN-D [18]	52.50%	41.94%	73.91%	35%	50%	56.25%
	READ [15]	50%	**83.87%**	30.43%	25%	50%	43.75%

[1] Exclude *Unused* CAN signals since they comprise the largest portion of the 1-bit long signal. Each *Unused* signal is regarded as occupying only one bit position, even when they appear consecutively within a CAN frame.

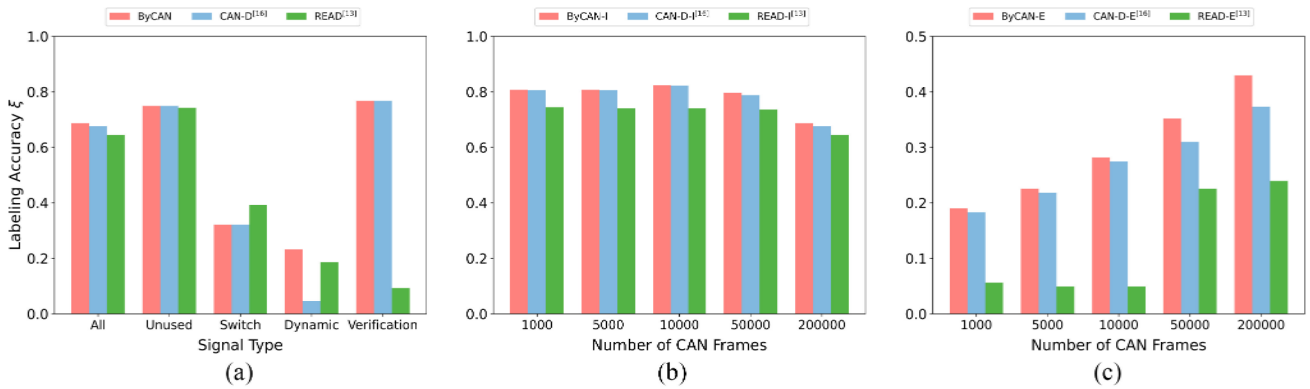

Fig. 5. Comparison of labeling accuracy of different systems. The y-axis is the labeling accuracy ξ for all subplots. The x-axis is the CAN signal type in (a) and the x-axis is the number of CAN frames in (b) and (c). Subplot (b) gives the labeling accuracy, including *Unused* CAN signals while subplot (c) gives the labeling accuracy excluding *Unused* CAN signals.

D. Labeling Performance

1) General Labeling: The labeling accuracy ξ for the general labels of CAN signals is discussed among different RE systems in Fig. 5. As indicated in Fig. 5, the ByCAN system that labels the CAN signals with multiple inputs (i.e., labeling threshold and flip rate of signal block) outperforms the READ and CAN-D system which use only bit-flip rate and multiple bit-level features, respectively. The general labels of CAN signals are *Unused*, *Switch*, *Dynamic*, and *Verification*. In Fig. 5, the impact of different CAN signal types and the number of CAN frames on ξ are discussed.

The overall labeling accuracy ξ of ByCAN is 1.09% slightly greater than that of CAN-D and 4.23% higher than that of READ. The performances of the three different RE systems have little difference for labeling the *Unused* CAN signals. The ByCAN outperforms CAN-D and READ except in labeling the *Switch* CAN signals. The READ system achieves 39.29% labeling accuracy for the *Switch* CAN signals, while the ByCAN and CAN-D have the same labeling accuracy of 32.14%. READ has higher labeling accuracy for the *Switch* CAN signals due to the excessively sliced small boundaries at the bit level that lead to more CAN signals identified as *Switch*. For READ, the tradeoff of the high labeling accuracy of the *Switch* (39.29%) and *Dynamic* (18.60%) CAN signals is the lower labeling accuracy of the *Verification* (9.3%) CAN signals. This discrepancy may be due to the READ solely using the bit-flip rate, which could cause mislabeling of *Verification* signals as *Dynamic* or *Switch*. This mislabeling is particularly likely when the bit in the most significant position (MSB) exhibits a significantly different bit-flip rate compared to the bit in the least significant position (LSB) of the *Verification* signals.

The impact of the number of CAN frames is demonstrated in Fig. 5(b) and (c). The labeling accuracy ξ of the assessments, including *Unused* CAN signals, slightly decreases with a larger size of CAN frames as indicated in Fig. 5(b). When $T_C = 10\,000$, the labeling accuracy of the ByCAN system, including *Unused* CAN signals (i.e., ByCAN-I) is 82.38%, that of CAN-D (i.e., CAN-D-I) is 82.24%, and that of READ (i.e., READ-I) is 74.04%. The labeling accuracy of the ByCAN-I, CAN-D-I, and READ-I drops to 68.72%, 67.62%, and 64.48%, respectively. On the contrary, the labeling accuracy ξ of ByCAN-E, CAN-D-E, and READ-E increases largely from 28.17%, 27.46%, and 4.93% to 42.96%, 37.32%, and 22.54%, when the number of CAN frames increases from 10 000 to 200 000 as demonstrated in Fig. 5(c). The features of CAN signals become more accurate with more CAN frames, which improves the labeling accuracy for *Switch*, *Dynamic*, and *Verification* CAN signals and reduces the false positive rate of *Unused* CAN signals. Based on our experiments, the optimum of T_C for the labeling accuracy is 10 000.

Compared to the labeling accuracy in Fig. 5(a) which counts all the CAN signals, including those rarely triggered signals during data collection, the labeling accuracy in Table VI evaluates the precision of labeling performance for the triggered CAN signals. The *Switch* and *Dynamic* CAN signal types are highlighted due to the significant differences in labeling accuracy when using triggered signals versus all the CAN signals from the ground truth. Not all *Switch* and *Dynamic* CAN signals can be triggered during the experiments, leading

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
LABELING ACCURACY OF TRIGGERED SIGNALS

Type of Triggered Signals	Labeling Accuracy ξ[1]
Switch	94.74%
Dynamic	70.59%

[1] ξ is estimated specifically for the CAN signals that are triggered during the experiments.

to the low labeling accuracy. The labeling accuracy increases from 32.14% in Fig. 5(a) to 94.74% in Table VI for the *Switch* signals. The same trend is found for the *Dynamic* CAN signals whose labeling accuracy increases from 23.26% to 70.59%. The result shows a good labeling performance of ByCAN for all the triggered and collected CAN signals.

2) Descriptive Labeling: ByCAN leverages the template matching to find the similarity between the sliced CAN signals and the templates of the OBD diagnostic data to associate them with descriptive meanings. In the proposed system, only the *Dynamic* CAN signals are further processed in this step to efficiently identify the descriptive labels with minimum time cost. ByCAN first introduces the DTW algorithm in labeling the descriptive labels to the CAN signals with the templates of the OBD diagnostic data. By using DTW, the small time shift problem between the collected CAN frames and the OBD diagnostic data can be solved.

Within the descriptive labels that can be extracted from the OBD diagnostic data, the engine speed, vehicle speed, throttle position, fuel rail gauge pressure, and calculated engine load stand out as the target labels that have potential significance for subsequent assessments conducted by industry engineers. Few *Dynamic* CAN signals are sliced due to the limited number of known functions to be triggered. However, ByCAN still shows a high labeling accuracy of identifying descriptive labels for the sliced *Dynamic* CAN signals. In our experiments, all the signals matched as the engine speed and throttle position labels are correctly categorized, and the labeling accuracy is drop to 50% for those with the vehicle speed label. Although ByCAN does not have the same level of labeling accuracy for nonspeed-related labels, it is worth noting that the speed-related labels have valuable insights and guidance for the future research.

VI. CONCLUSION

In this article, we proposed the fully automated RE system named ByCAN to decode the specification of the CAN frames without prior knowledge. Based on the observation of DBC files and CAN frames, CAN signal patterns are identified and used in ByCAN for the RE CAN signals. ByCAN is the first to introduce multiple byte-level and bit-level features that capture the characteristics and patterns of CAN signals to slice and label CAN signals. ByCAN applies the byte-level clustering with the DBSCAN algorithm for enhancing the slicing and labeling performance. The proposed ByCAN system is also the first to use the DTW algorithm in matching sliced *Dynamic* CAN signals with the templates of OBD diagnostic data for associating CAN signals with descriptive labels facilitating further research. The experiments validated that ByCAN enhances the performance of CAN signal slicing and labeling with the real-world data from various cars. ByCAN requires only a short CAN trace in our experiments to achieve a good reverse-engineering result.

ACKNOWLEDGMENT

The authors thank their industry partner IAG for providing access to test vehicles and Thanh Phuoc Nguyen for helping with technical issues and experiments.

REFERENCES

[1] G. Leen and D. Heffernan, "Expanding automotive electronic systems," *Computer*, vol. 35, no. 1, pp. 88–93, 2002.
[2] W. Wu et al., "A survey of intrusion detection for in-vehicle networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 3, pp. 919–933, Mar. 2020.
[3] M. K. Ishak and F. K. Khan, "Unique message authentication security approach based controller area network (CAN) for anti-lock braking system (ABS) in vehicle network," *Procedia Comput. Sci.*, vol. 160, pp. 93–100, Nov. 2019. [Online]. Available: https://www.sciencedirect.com/journal/procediacomputer-science/vol/160/suppl/C
[4] W. Zeng, M. A. Khalid, and S. Chowdhury, "In-vehicle networks outlook: Achievements and challenges," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 3, pp. 1552–1571, 3rd Quart., 2016.
[5] O. Avatefipour and H. Malik, "State-of-the-art survey on in-vehicle network communication (CAN-bus) security and vulnerabilities," 2018, *arXiv:1802.01725*.
[6] A. Buscemi, I. Turcanu, G. Castignani, A. Panchenko, T. Engel, and K. G. Shin, "A survey on controller area network reverse engineering," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 3, pp. 1445–1481, 3rd Quart., 2023.
[7] B. Ma, X. Wang, W. Ni, and R. P. Liu, "Personalized location privacy with road network-indistinguishability," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 20860–20872, Nov. 2022.
[8] B. Ma et al., "New cloaking region obfuscation for road network-indistinguishability and location privacy," in *Proc. 25th Int. Symp. Res. Attacks, Intrus. Defenses*, 2022, pp. 160–170.
[9] B. Ma et al., "Vehicle trajectory obfuscation and detection," in *Cybersecurity for Smart Cities: Practices and Challenges*. Cham, Switzerland: Springer, 2023, pp. 121–134.
[10] B. Ma et al., "Location privacy threats and protections in future vehicular networks: A comprehensive review," 2023, *arXiv:2305.04503*.
[11] B. C. Nolan, S. Graham, B. Mullins, and C. S. Kabban, "Unsupervised time series extraction from controller area network payloads," in *Proc. IEEE 88th Veh. Technol. Conf. (VTC)*, 2018, pp. 1–5.
[12] D. Frassinelli, S. Park, and S. Nürnberger, "I know where you parked last summer: Automated reverse engineering and privacy analysis of modern cars," in *Proc. IEEE Symp. Security Privacy (SP)*, 2020, pp. 1401–1415.
[13] M. Verma, R. Bridges, and S. Hollifield, "ACTT: Automotive CAN tokenization and translation," in *Proc. Int. Conf. Comput. Sci. Comput. Intell. (CSCI)*, 2018, pp. 278–283.
[14] M. Markovitz and A. Wool, "Field classification, modeling and anomaly detection in unknown CAN bus networks," *Veh. Commun.*, vol. 9, pp. 43–52, Jul. 2017.
[15] M. Marchetti and D. Stabili, "READ: Reverse engineering of automotive data frames," *IEEE Trans. Inf. Forensics Security*, vol. 14, pp. 1083–1097, 2018.
[16] M. D. Pesé, T. Stacer, C. A. Campos, E. Newberry, D. Chen, and K. G. Shin, "LibreCAN: Automated CAN message translator," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2019, pp. 2283–2300.
[17] A. Buscemi, I. Turcanu, G. Castignani, R. Crunelle, and T. Engel, "CANMatch: A fully automated tool for CAN bus reverse engineering based on frame matching," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 12358–12373, Dec. 2021.
[18] M. E. Verma, R. A. Bridges, J. J. Sosnowski, S. C. Hollifield, and M. D. Iannacone, "CAN-D: A modular four-step pipeline for comprehensively decoding controller area network data," 2020, *arXiv:2006.05993*.
[19] W. Choi, S. Lee, K. Joo, H. J. Jo, and D. H. Lee, "An enhanced method for reverse engineering CAN data payload," *IEEE Trans. Veh. Technol.*, vol. 70, no. 4, pp. 3371–3381, Apr. 2021.

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

[20] B. Blaauwendraad and V. Kieberl. "Automated reverse-engineering of CAN messages using OBD-II and correlation coefficients." 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:221067722

[21] "Comma.ai/OpenDBC." 2017. [Online]. Available: https://github.com/commaai/opendbc

[22] G. Hermawan and E. Husni, "Acquisition, modeling, and evaluating method of driving behavior based on OBD-II: A literature survey," in *IOP Conf. Ser.: Materials Sci. Eng.*, vol. 879, no. 1, 2020, Art. no. 012030.

[23] K. McCord, *Automotive Diagnostic Systems: Understanding OBD I and OBD II*. North Branch, MN, USA: CarTech Inc., 2011.

[24] Y. Zhang, B. Ge, X. Li, B. Shi, and B. Li, "Controlling a car through OBD injection," in *Proc. IEEE 3rd Int. Conf. Cyber Security Cloud Comput. (CSCloud)*, 2016, pp. 26–29.

[25] M. Ammar, H. Janjua, A. S. Thangarajan, B. Crispo, and D. Hughes, "Securing the on-board diagnostics port (OBD-II) in vehicles," *SAE Int. J. Transp. Cybersecurity Privacy*, vol. 2, no. 2, pp. 83–106, 2020.

[26] C. Young, J. Svoboda, and J. Zambreno, "Towards reverse engineering controller area network messages using machine learning," in *Proc. IEEE 6th World Forum Internet Things (WF-IoT)*, 2020, pp. 1–6.

[27] S. Rajapaksha, H. Kalutarage, M. O. Al-Kadri, A. Petrovski, G. Madzudzo, and M. Cheah, "AI-based intrusion detection systems for in-vehicle networks: A survey," *ACM Comput. Surv.*, vol. 55, no. 11, pp. 1–40, 2023.

[28] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. KDD*, 1996, pp. 226–231.

[29] E. Schubert, J. Sander, M. Ester, H. P. Kriegel, and X. Xu, "DBSCAN revisited, revisited: Why and how you should (still) use DBSCAN," *ACM Trans. Database Syst. (TODS)*, vol. 42, no. 3, pp. 1–21, 2017.

[30] S. Checkoway et al., "Comprehensive experimental analyses of automotive attack surfaces," in *Proc. 20th USENIX Security Symp. (USENIX Security)*, 2011, pp. 1–16.

[31] S. Woo, H. J. Jo, and D. H. Lee, "A practical wireless attack on the connected car and security protocol for in-vehicle CAN," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 2, pp. 993–1006, Apr. 2015.

[32] A. Buscemi, G. Castignani, T. Engel, and I. Turcanu, "A data-driven minimal approach for CAN bus reverse engineering," in *Proc. IEEE 3rd Connect. Autom. Veh. Symp. (CAVS)*, 2020, pp. 1–5.

[33] T. U. Kang, H. M. Song, S. Jeong, and H. K. Kim, "Automated reverse engineering and attack for CAN using OBD-II," in *Proc. IEEE 88th Veh. Technol. Conf. (VTC)*, 2018, pp. 1–7.

[34] J. de Hoog, T. Bogaerts, W. Casteels, S. Mercelis, and P. Hellinckx, "Online reverse engineering of CAN data," *Internet Things*, vol. 11, Sep. 2020, Art. no. 100232.

[35] S. Salvador and P. Chan, "Toward accurate dynamic time warping in linear time and space," *Intell. Data Anal.*, vol. 11, no. 5, pp. 561–580, 2007.

[36] H. Wen, Q. Zhao, Q. A. Chen, and Z. Lin, "Automated cross-platform reverse engineering of CAN bus commands from mobile apps," in *Proc. Netw. Distrib. Syst. Security Symp.*, 2020, pp. 1–17.

[37] Z. Bi, G. Xu, G. Xu, C. Wang, and S. Zhang, "Bit-level automotive controller area network message reverse framework based on linear regression," *Sensors*, vol. 22, no. 3, p. 981, 2022.

[38] L. Liang et al., "Leveraging byte-level features for LSTM-based anomaly detection in controller area network," in *Proc. IEEE Global Commun. Conf.*, 2022, pp. 4903–4908.

[39] X. Lin, B. Ma, X. Wang, Y. He, R. P. Liu, and W. Ni, "Multi-layer reverse engineering system for vehicular controller area network messages," in *Proc. IEEE 25th Int. Conf. Comput. Support. Coop. Work Design (CSCWD)*, 2022, pp. 1185–1190.

[40] D. J. Berndt and J. Clifford, "Using dynamic time warping to find patterns in time series," in *Proc. KDD Workshop*, 1994, pp. 359–370.

Xiaojie Lin received the M.I.T. degree from the Faculty of Engineering and Information Technology, University of Technology Sydney, Sydney, NSW, Australia, in 2020, where she is currently pursuing the Ph.D. degree.

Her main research interests include cybersecurity, privacy, and machine learning.

Baihe Ma received the B.E. and M.E. degrees from Xidian University, Xi'an, China, in 2016 and 2019, respectively, and the Ph.D. degree from the University Technology Sydney, Sydney, NSW, Australia, in 2024.

His research interests include cybersecurity, data privacy, location privacy, and machine learning.

Xu Wang (Member, IEEE) received the B.E. degree from Beijing Information Science and Technology University, Beijing, China, in 2010, the first Ph.D. degree from Beijing University of Posts and Telecommunications in 2019, and the second Ph.D. degree from the University of Technology Sydney, Sydney, NSW, Australia, in 2020.

He is currently a Senior Lecturer with the School of Electrical and Data Engineering, University of Technology Sydney. His research interests include cybersecurity, blockchain, privacy, and network dynamics.

Guangsheng Yu received the B.Sc. and M.Sc degrees from the University of New South Wales, Sydney, NSW, Australia, from 2011 to 2015, and the Ph.D. degree from the Faculty of Engineering and Information Technology, University of Technology Sydney (UTS), Sydney, in 2021.

He is currently a Postdoctoral Research Fellow with CSIRO Data61, Eveleigh, NSW, and an Adjunct Lecturer with UTS. His main research interests lie in cybersecurity, blockchain, and distributed learning.

Ying He (Senior Member, IEEE) received the B.Eng. degree in telecommunications engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2009, and the Ph.D. degree in telecommunications engineering from the University of Technology Sydney, Sydney, NSW, Australia, in 2017.

She is currently a Senior Lecturer with the School of Electrical and Data Engineering, University of Technology Sydney. Her research interests are physical–layer algorithms in wireless communication with machine learning, vehicular communication, spectrum sharing, and satellite communication.

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.

Ren Ping Liu (Senior Member, IEEE) received the B.E. degree from Beijing University of Posts and Telecommunications, Beijing, China, and the Ph.D. degree from the University of Newcastle, Callaghan, NSW, Australia, in 1985 and 1996, respectively.

He is a Professor and the Head of the Discipline of Network and Cybersecurity, University of Technology Sydney, Sydney, NSW. As a Research Leader, a Certified Network Professional, and a full-stack Web Developer, he has delivered networking and cybersecurity solutions to government agencies and industry customers. He has supervised over 30 Ph.D. students and has over 200 research publications. His research interests include wireless networking, 5G, IoT, vehicular networks, 6G, cybersecurity, and blockchain.

Prof. Liu was awarded the Australian Engineering Innovation Award in 2012 and the CSIRO Chairman's Medal for contributing to the Wireless Backhaul Project. He was the Winner of the NSW iAwards 2020 for leading the Blockchain-Enabled Fish Provenance And Quality Tracking Project. He was the Founding Chair of the IEEE NSW VTS Chapter.

Wei Ni (Fellow, IEEE) received the B.E. and Ph.D. degrees in electronic engineering from Fudan University, Shanghai, China, in 2000 and 2005, respectively.

He is currently a Team Leader with CSIRO, Sydney, NSW, Australia, and an Adjunct Professor with the University of Technology Sydney, Sydney. He was a Postdoctoral Research Fellow with Shanghai Jiaotong University, Shanghai, from 2005 to 2008, the Deputy Project Manager of the Bell Labs Research and Innovation Center, Alcatel/Alcatel-Lucent, Shanghai, from 2005 to 2008, and a Senior Researcher with Devices Research and Development, Nokia, Shanghai, from 2008 to 2009. His research interests include stochastic optimization, game theory, graph theory, and their applications to network and security.

Authorized licensed use limited to: Högskolan Väst. Downloaded on February 25,2026 at 10:33:21 UTC from IEEE Xplore. Restrictions apply.